CHASE.AI: Agentic Contract-Based Multilevel Engineering for High-Assurance Complex Systems

PI: Pierluigi Nuzzo, Associate Professor, Department of Electrical Engineering and Computer Sciences, University of California, Berkeley

Cash funding needed: $80,000
AWS Promotional Credits needed: $25,000–$40,000

Amazon contact (optional): [Amazon contact name], [email]

Abstract - Large language model (LLM) agents are increasingly employed to design complex systems, such as distributed cyber-physical systems, multi-robot systems, and hardware. However, AI-generated designs often contain errors: agents split a design into parts that do not fit together, drift from their instructions over long design tasks, and leave no record that allows an error to be traced back to its cause. In this project, we propose a framework, termed CHASE.AI, for sound and traceable decomposition of complex systems based on assume-guarantee (A/G) contracts. In CHASE.AI, agents propose a contract for each component at every level of a design hierarchy, and a contract engine accepts a step only when it certifies that the components together meet their parent's requirements. CHASE.AI follows a three-pronged approach. First, it extends contract theory so that every decomposition step, even across formalisms and abstraction levels, can be checked by existing verification and synthesis tools. Second, it develops an agentic decomposition search guided by a library of certified components, in which only the tools that verify the smallest components change across domains. Finally, it lets the agents learn from formal feedback, where counterexamples repair decompositions and serve as verifiable rewards to post-train the proposing models. CHASE.AI will extend CHASE, our open-source contract-based design framework, with LLM agents and will be evaluated on distributed systems, multi-robot systems, and hardware.

Keywords – assume-guarantee contracts, compositional verification, distributed systems, GR(1), neurosymbolic reasoning, LLM agents, reinforcement learning with verifiable rewards, hardware formal verification.

Introduction

LLM agents are increasingly employed to design complex systems, from distributed cyber-physical systems, such as an automatic emergency braking system whose networked components must trade consistency against availability [1], to multi-robot systems and custom silicon. Agents translate natural-language requirements of robotic missions into temporal logic [2] and generate hardware assertions [3, 4] faster than design teams can review them. However, AI-generated designs often contain errors, and these errors concentrate in decomposition, which every complex design requires. In our experience, agents that decompose end-to-end hardware properties may report timed-out runs as proofs. Over long design tasks, agents that delegate subtasks or compact their context restate the requirements in natural language and drift away from them [5]. Moreover, the reasoning behind a design is buried in transcripts, so a late error cannot be traced to its cause. As a result, AI-driven design relies today on trial and error and on testing the final artifact. A new generation of design methods would be essential to make every step of AI-driven design traceable and checkable, and to keep the overall design correct as it is decomposed across components and abstraction levels.

Contract-based design provides a formal basis for this goal. An A/G contract specifies what a component guarantees under assumptions on its environment [6]. A decomposition is correct when the composition of the component contracts refines the system contract, that is, when the components together guarantee at least what the system promises under no stronger assumptions [7, 8]. Existing contract-based tools, including CHASE [9], Pacti [10], and OCRA [11], assume that contracts are written by human designers, and existing decomposition methods for synthesis rely on a fixed library of components [12]. Decomposition is also used by recent LLM reasoning methods: language models search over decompositions [13], compose

functions validated by tests [14], and delegate subgoals to automated provers [15, 16, 17]. However, their checks are either informal, such as tests and verbal reflection [18], or limited to proof goals, and do not establish that the components of a system are realizable, i.e., can be implemented, and compose correctly. Letting agents write the contracts raises three scientific questions. First, the levels of a design are often described in different formalisms, so each decomposition step must remain checkable by existing verification and synthesis tools. Second, the existence of a decomposition into realizable components is undecidable for general architectures [19], so the search must never lose certified progress. Third, an agent may search indefinitely unless each failure is explained in a form that excludes whole families of candidates.

The proposed framework, termed CHASE.AI, addresses these challenges via a three-pronged approach. The central principle of CHASE.AI is that agents propose and contracts certify: a step proposed by an agent is accepted only with a certificate that a solver can check, so that guarantees established at higher abstraction levels propagate to the implementation. First, CHASE.AI extends contract theory so that decomposition in space, across abstraction levels, and over time can be checked by existing tools (Thrust 1). Second, it develops an agentic decomposition search in which LLM agents, guided by a library of certified components, propose contracts at every level (Thrust 2). Finally, it learns from formal feedback by repairing failed decompositions and growing libraries of certified contracts and repair patterns (Thrust 3). CHASE.AI builds on our prior work on contract-based design methodologies [20] and extends the CHASE framework [9], which provides a contract algebra with interfaces to verification and synthesis tools and to LLM agents. Our work on contract-based architecture exploration [21, 22] has shown that infeasibility certificates can be generalized via subgraph isomorphism to prune large design spaces, and received the Best Paper Award at DATE 2024. Our timing contracts enabled consistency and availability analysis of distributed systems [1], while ACIES [23] and our ongoing work apply agent-proposed contracts to multi-robot control and register-transfer level (RTL) verification. If successful, this project will lay the foundations for AI-driven design of complex systems in which every step is certified by a solver, errors are traced to their cause, and components are reused.

Methods

The core of CHASE.AI is a neurosymbolic loop: LLM agents propose decompositions (Thrust 2), a contract engine certifies them on the theory of Thrust 1, and its verdicts return to the agents in context and as post-training rewards (Thrust 3). We will evaluate it on distributed systems, multi-robot control, and hardware.

Thrust 1: Contract Theory for Decomposition. CHASE.AI decomposes a design along three dimensions. Spatial decomposition splits a system into interacting components, as in our work on architectures [21], distributed systems [1], and multi-robot control [23]. Hierarchical decomposition refines requirements across abstraction levels that use different formalisms, which we relate through contract embeddings based on conservative approximations [24], which map behaviors between formalisms so that a refinement proved on the embedded contracts also holds for the original ones. Temporal decomposition splits a mission into contracts on non-overlapping time intervals, each enforced by a time-varying control barrier function, i.e., a certificate that keeps the system inside the states allowed by the contract [25]. The contracts themselves are domain-specific. Timing contracts for distributed systems associate the logical ordering of events with execution and communication latencies, and bound the inconsistency and unavailability of each component [1]. Robot contracts couple a mission layer in GR(1), a fragment of linear temporal logic that admits efficient controller synthesis [26], with controllers certified by control barrier functions [27] or reachability analysis. Learned policies, such as vision-language-action (VLA) models that map camera images and language instructions to robot actions [28], are guarded at run time by safety filters and monitors that report violated assumptions to the decomposition, and are post-trained with rewards derived from their component contracts. Hardware contracts relate the ports of register-transfer-level modules over clock cycles and may refer to auxiliary (ghost) state that is added for verification and checked on the design.

Thrust 2: Agentic Decomposition Search. CHASE.AI will represent a design as a hierarchy of contracts that

LLM agents build recursively, starting from the system requirements. At each level, an agent proposes contracts for the subcomponents, guided by certified decompositions of similar components retrieved from the contract library, and the engine accepts the proposal only if the subcomponents together are certified to meet the parent contract. Each accepted subcomponent is either verified directly by a back end or handed to a new agent that works only on its own contract, which keeps the context of every agent small as the design grows. When a component cannot meet its contract, the failure is reported to its parent, which revises the decomposition while keeping the certified parts. Alternative decompositions will be explored by a search in which the agents propose candidates and the engine prunes them. Only the tools that verify the leaves of the hierarchy change across domains, e.g., mixed-integer linear programming for timing contracts, synthesis and reachability tools for robot controllers, and model checkers for hardware modules.

Thrust 3: Learning from Formal Feedback. Every failed check returns a diagnosis that points to the responsible contract terms: a counterexample trace when refinement or model checking fails, a minimal set of conflicting terms (an unsatisfiable core) when contracts are inconsistent, or vacuous because their assumptions can never hold [29], and a minimal set of requirements that no controller can meet together (an unrealizable core) when synthesis fails. Agents call these operations through a Model Context Protocol (MCP) server, a standard interface for tool use, while no LLM is part of the engine. Repairs are then attempted in three ways. Formal repair adjusts contract parameters directly, e.g., by tuning inconsistency and unavailability bounds as in [1]. Agent repair returns the diagnosis to the agent. Pattern-based repair generalizes recurring failures into repair patterns that, through subgraph isomorphism [21], exclude every candidate containing the same failing structure, and records them in a pattern library. Certified contracts are stored in a contract library, so that the framework evolves with each design: the decomposer retrieves certified decompositions from it, while search and verification reuse the pattern library. Finally, certified decompositions are used to post-train an open-weight proposer, with the checks of the engine as verifiable rewards, i.e., rewards computed by a checker, graded by the consistency, refinement, and realizability of each component contract. As the engine checks every proposal, soundness never depends on the proposer.

Evaluation and preliminary results. Our prior work on auto-formalization [30, 31] improves its reliability by screening diverse candidate formalizations with solvers, a principle that CHASE.AI applies to contract generation and revision. For distributed systems, our timing contracts [1] verified and repaired the consistency and availability requirements of an automatic emergency braking system and a multi-drone coordination scenario. In multi-robot control, ACIES [23] decomposes mission requirements into GR(1) contracts, synthesizes controllers, and executes them under control barrier functions with runtime monitors, reusing certified components from its contract library. In hardware, component contracts are SystemVerilog assertions (SVAs) with ghost-state models, discharged by hardware model checkers. Our prototype certified an end-to-end in-order delivery property of an asynchronous link from the HierSVA benchmark [32] with 25 model-checking runs and about 16,000 LLM input tokens, while direct model checking of the end-to-end property did not finish within a considerably longer time limit. The decomposition found at FIFO depth 8 certified depths 16 to 512 with no further LLM calls, and an agent without the protocol used 24 times as many tokens without completing the proof. We will compare frontier models with and without formal feedback and retrieval against the post-trained proposer, measuring the requirements certified beyond direct verification or synthesis, the acceptance rate, and solver runs and tokens per certificate.

Expected results

Months	Research result	Deliverable
1–3	Decomposition agent with formal feedback, contract embeddings across formalisms	CHASE release with MCP server and agent skill, technical report

4–6	Formal, agent, and pattern-based repair, contract and pattern libraries	RTL benchmark with faulty variants, theory paper (e.g., CAV)
7–9	Agentic decomposition search in all three domains, dataset of certified decompositions	Evaluation, open dataset, paper (e.g., FMCAD)
10–12	Proposer and VLA policies post-trained with contract-based rewards	Open-weight proposer, paper (e.g., NeurIPS), tutorial, talks to Amazon

Beyond the first year, certified decompositions will grow into a training corpus for a cross-domain decomposition model, and the benchmark, dataset, tutorial, code, and run traces will be released openly.

Funds needed

Cash funding ($80,000). Partial support of one graduate student researcher (stipend and fees, [amount]), and travel to present the results at conferences ([amount]). No indirect expenses are included.

AWS Promotional Credits ($25,000–$40,000). Amazon Bedrock for agent inference, Amazon SageMaker GPU instances for post-training an open-weight decomposition proposer (Thrust 3), Amazon EC2 instances for parallel runs of nuXmv, slugs, and SymbiYosys, which also compute the rewards during post-training, and Amazon S3 for benchmarks and run traces. Commercial verification tools use campus licenses.

Additional information

1. Application domain and analysis techniques. Compositional design of distributed systems, multi-robot systems, and hardware, using A/G contracts, mixed-integer linear programming (Gurobi), linear temporal logic (LTL) model checking (nuXmv), satisfiability modulo theories (SMT) reasoning, GR(1) synthesis (slugs), and hardware model checking (SymbiYosys, Jasper). It covers sound neurosymbolic reasoning, LLM–solver interfaces, vacuity and inconsistency detection, distributed systems, and formal models of hardware.

2. Current applications. CHASE has been used for requirement formalization and validation of cyber-physical systems [9], contract-based architecture exploration [21, 22], and consistency and availability analysis of distributed systems with Lockheed Martin [1]. Our RTL flow runs on OpenTitan and HierSVA designs.

3. Potential applications to Amazon. (i) Distributed cloud services, where end-to-end latency, availability, and consistency budgets can be decomposed into per-component timing contracts [1], complementing protocol-level models in TLA+ or P. (ii) Amazon Robotics fleets, where missions can be decomposed hierarchically into contracts for small groups of robots [23], and learned VLA policies can be guarded at run time by contract monitors and post-trained with contract-based rewards. (iii) Verification of AWS processors and accelerators, where end-to-end properties of blocks such as interconnects and coherence controllers that exceed direct model checking can be decomposed into module-level SVA contracts, reused when a block is resized.

4–5. Prior Amazon Research Awards. No, and no Amazon funding or AWS credits in the past three years.

6. Assumptions and soundness. A certified result is sound when (i) the back ends return correct verdicts, (ii) the translation into each back end preserves the semantics of the contracts, (iii) the embeddings between formalisms are conservative approximations, and (iv) the top-level specification is correct, so agent-written contracts are checked for consistency and vacuity. Learned components guarded only at run time are excluded from the certificate. Bounded results never count as proofs, and LLM outputs are never trusted.

7. Tools. *Code:* github.com/chase-cps/chase and github.com/chase-cps/chase-mcp. *License:* BSD 3-Clause for CHASE and its MCP server. *On-boarding:* a website (chase-cps.github.io) with a usage guide and API reference, a Docker image, and an agent skill for LLM agents. A tutorial is planned. *Maintenance:* actively maintained, with over 250 commits to CHASE and its MCP server in three months, and CHASE has three contributors.

Appendix A – References

[1] Xiao, Yifeng, et al. “Contract-Based Consistency and Availability Analysis for Distributed Cyber-Physical Systems.” MEMOCODE. 2026.

[2] Cosler, Matthias, et al. “nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language Models.” CAV. 2023.

[3] Orenes-Vera, Marcelo, et al. “AutoSVA: Democratizing Formal Verification of RTL Module Interactions.” DAC. 2021.

[4] Kang, Minwoo, et al. “FVEval: Understanding Language Model Capabilities in Formal Verification of Digital Hardware.” arXiv:2410.23299. 2024.

[5] Cemri, Mert, et al. “Why Do Multi-Agent LLM Systems Fail?” arXiv:2503.13657. 2025.

[6] Benveniste, Albert, et al. “Contracts for System Design.” Foundations and Trends in Electronic Design Automation. 2018.

[7] Incer, Inigo, Albert Benveniste, and Alberto Sangiovanni-Vincentelli. “Some Algebraic Aspects of Assume-Guarantee Reasoning.” arXiv:2309.08875. 2023.

[8] Incer, Inigo. “The Algebra of Contracts.” Ph.D. thesis, UC Berkeley, EECS-2022-99. 2022.

[9] Nuzzo, Pierluigi, et al. “CHASE: Contract-Based Requirement Engineering for Cyber-Physical System Design.” DATE. 2018.

[10] Incer, Inigo, et al. “Pacti: Scaling Assume-Guarantee Reasoning for System Analysis and Design.” arXiv:2303.17751. 2023.

[11] Cimatti, Alessandro, Michele Dorigatti, and Stefano Tonetta. “OCRA: A Tool for Checking the Refinement of Temporal Contracts.” ASE. 2013.

[12] Iannopollo, Antonio, Stavros Tripakis, and Alberto Sangiovanni-Vincentelli. “Specification Decomposition for Synthesis from Libraries of LTL Assume/Guarantee Contracts.” DATE. 2018.

[13] Yao, Shunyu, et al. “Tree of Thoughts: Deliberate Problem Solving with Large Language Models.” NeurIPS. 2023.

[14] Zelikman, Eric, et al. “Parsel: Algorithmic Reasoning with Language Models by Composing Decompositions.” NeurIPS. 2023.

[15] Jiang, Albert Q., et al. “Thor: Wielding Hammers to Integrate Language Models and Automated Theorem Provers.” NeurIPS. 2022.

[16] Yang, Kaiyu, et al. “LeanDojo: Theorem Proving with Retrieval-Augmented Language Models.” NeurIPS. 2023.

[17] Ren, Z. Z., et al. “DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition.” arXiv:2504.21801. 2025.

[18] Shinn, Noah, et al. “Reflexion: Language Agents with Verbal Reinforcement Learning.” NeurIPS. 2023.

[19] Pnueli, Amir, and Roni Rosner. “Distributed Reactive Systems Are Hard to Synthesize.” FOCS. 1990.

[20] Nuzzo, Pierluigi, et al. “A Platform-Based Design Methodology With Contracts and Related Tools for the Design of Cyber-Physical Systems.” Proceedings of the IEEE. 2015.

[21] Xiao, Yifeng, et al. “Efficient Exploration of Cyber-Physical System Architectures Using Contracts and Subgraph Isomorphism.” DATE. 2024.

[22] Xiao, Yifeng, and Pierluigi Nuzzo. “Contract-Based Architecture Exploration of Cyber-Physical Systems via Satisfiability Modulo Convex Programming.” DATE. 2026.

[23] Xiao, Yifeng, et al. “ACIES: A Self-Evolving Agentic Contract-Based Framework for Cyber-Physical System Control Synthesis.” REALM Workshop at EMNLP. 2026.

[24] Naik, Nikhil Vijay, Alessandro Pinto, and Pierluigi Nuzzo. “Contract-Based Hierarchical Modeling and Traceability of Heterogeneous Requirements.” IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems. 2024.

[25] Waqas, Muhammad, et al. “Control Barrier Function Contracts for Vehicular Mission Planning Under Signal Temporal Logic Specifications.” arXiv:2209.07478. 2022.

[26] Bloem, Roderick, et al. “Synthesis of Reactive(1) Designs.” Journal of Computer and System Sciences. 2012.

[27] Ames, Aaron D., et al. “Control Barrier Functions: Theory and Applications.” ECC. 2019.

[28] Kim, Moo Jin, et al. “OpenVLA: An Open-Source Vision-Language-Action Model.” CoRL. 2024.

[29] Maoz, Shahar, and Rafi Shalom. “Inherent Vacuity for GR(1) Specifications.” ESEC/FSE. 2020.

[30] Ni, Zhiyu, Yifeng Xiao, and Zheng Liang. “Agentified Assessment of Logical Reasoning Agents.” Agents in the Wild (AIWILD) Workshop at ICLR. 2026.

[31] Ni, Zhiyu, et al. “Draft-and-Prune: Improving the Reliability of Auto-formalization for Logical Reasoning.” arXiv:2603.17233. Code: github.com/zyni2001/draft-and-prune. 2026.

[32] Nie, Maohua, et al. “HierSVA: A Data Synthesis Pipeline, Dataset, and Benchmark for LLM-Driven Hierarchical Hardware Formal Verification.” arXiv:2606.13706. 2026.